



**Rajshahi University of Engineering & Technology
(RUET)**

Department of Electrical & Computer Engineering

Lab Report - 1

**Course Title: Discrete Mathematics & Numerical Methods
Course Code: ECE 2214**

Experiment Name:

Implementation of root finding by Bisection Method

Submitted by,
Md. Yousuf
Roll : 2310023

Submitted to,
Md. Faysal Ahamed
Assistant Professor
Dept of ECE, RUET

Contents

1	Theory	2
2	Required Tools	2
3	Equations to Solve	2
4	Algorithm: Bisection Method	2
5	Code Implementation	3
5.1	Code for Equation 1	3
5.2	Code for Equation 2	4
6	Output and Graphs	5
6.1	Program Outputs (Command Window)	5
6.2	Graphical Representation	6
7	Discussion	7
8	Conclusion	7

List of Figures

1	Octave Terminal Output and Iteration Table for Equation 1	5
2	Octave Terminal Output and Iteration Table for Equation 2	6
3	Plot of Equation 1: $f(x) = x^3 - 2x^2 - 5$	6
4	Plot of Equation 2: $f(x) = e^x - 4\sin(x) - 3\ln(x)$	7

1 Theory

The bisection method is an iterative root-finding technique based on the Intermediate Value Theorem. It works for a continuous function $f(x)$ on $[a, b]$ where $f(a)$ and $f(b)$ have opposite signs, ensuring a root exists between them.

At each step, the midpoint is calculated as:

$$c = \frac{a + b}{2} \quad (1)$$

Then:

- If $f(a) \times f(c) < 0$, the root lies in $[a, c] \rightarrow$ set $b = c$.
- If $f(a) \times f(c) > 0$, the root lies in $[c, b] \rightarrow$ set $a = c$.

This process repeats until the interval is sufficiently small or the desired accuracy is achieved.

2 Required Tools

1. Mac M4 Air (Hardware)
2. GNU Octave (Software environment)
3. MS Office / LaTeX (For report generation)

3 Equations to Solve

To demonstrate the method, we solve two distinct non-linear equations:

- **Equation 1 (Polynomial):** $f(x) = x^3 - 2x^2 - 5 = 0$
- **Equation 2 (Logarithmic, Trigonometric, Exponential):** $f(x) = e^x - 4 \sin(x) - 3 \ln(x) = 0$

4 Algorithm: Bisection Method

1. Start with initial guesses a and b .
1. Input: equation $y(x)$, initial guesses a and b .
1. Set tolerance $lim = 0.001$.
1. Compute $fa = y(a)$ and $fb = y(b)$.

1. If $fa \times fb > 0$, display “Wrong guess” and stopspan₉(*start_span*)[*span*₉](*end_span*).

1. Else:

(a) Compute midpoint $c = (a + b)/2$ span₁₀(*start_span*)[*span*₁₀](*end_span*).

(a) Evaluate $fc = y(c)$ span₁₁(*start_span*)[*span*₁₁](*end_span*).

(a) While $|fc| > \text{lim}$:

- Print a, b, c, fc span₁₂(*start_span*)[*span*₁₂](*end_span*).
- If $fa \times fc < 0$, set $b = c$ else $a = c$ span₁₃(*start_span*)[*span*₁₃](*end_span*).
- Update $c = (a + b)/2$ and $fc = y(c)$ span₁₄(*start_span*)[*span*₁₄](*end_span*).

(b) Print root = c span₁₅(*start_span*)[*span*₁₅](*end_span*).

2. Stopspan₁₆(*start_span*)[*span*₁₆](*end_span*)

5 Code Implementation

Two separate and independent Octave scripts were developed to find the roots of the given equations.

5.1 Code for Equation 1

```

1 % Bisection Method for Equation 1: f(x) = x^3 - 2x^2 - 5
2 f = @(x) x^3 - 2*x^2 - 5;
3 a = 2; b = 3; % Initial guesses
4 lim = 0.001; % Tolerance limit
5
6 fa = f(a);
7 fb = f(b);
8
9 if fa * fb > 0
10     disp('Wrong guess');
11 else
12     fprintf('%-10s%-10s%-10s%-10s\n', 'a', 'b', 'c', 'f(c)');
13     fprintf('-----\n');
14
15     c = (a + b) / 2;
16     fc = f(c);
17
18     while abs(fc) > lim
19         fprintf('%-10.4f%-10.4f%-10.4f%-10.4f\n', a, b, c, fc)
20         ;
21         if fa * fc < 0
22             b = c;
23         else

```

```

24         a = c;
25         fa = f(a); % Update fa as a has changed
26     end
27
28     c = (a + b) / 2;
29     fc = f(c);
30 end
31
32 % Print final converging step
33 fprintf('%-10.4f%-10.4f%-10.4f%-10.4f\n', a, b, c, fc);
34 fprintf('Root for Equation 1 found at: %.4f\n', c);
35 end

```

Listing 1: Octave Code for $f(x) = x^3 - 2x^2 - 5$

5.2 Code for Equation 2

```

1  f = @(x) exp(x) - 4*sin(x) - 3*log(x);
2  a = 1; b = 2;
3  lim = 0.001;
4
5  fa = f(a);
6  fb = f(b);
7
8  if fa * fb > 0
9      disp('Wrong guess');
10 else
11     fprintf('%-10s%-10s%-10s%-10s\n', 'a', 'b', 'c', 'f(c)');
12     fprintf('-----\n');
13
14     c = (a + b) / 2;
15     fc = f(c);
16
17     while abs(fc) > lim
18         fprintf('%-10.4f%-10.4f%-10.4f%-10.4f\n', a, b, c, fc);
19         ;
20
21         if fa * fc < 0
22             b = c;
23         else
24             a = c;
25             fa = f(a);
26         end
27
28         c = (a + b) / 2;
29         fc = f(c);
30     end
31
32     fprintf('%-10.4f%-10.4f%-10.4f%-10.4f\n', a, b, c, fc);
33     fprintf('Root for Equation 2 found at: %.4f\n', c);
34 end

```

6 Output and Graphs

6.1 Program Outputs (Command Window)

The outputs obtained from the Octave terminal, showing the iteration tables and final roots for both equations, are attached below.

a	b	c	f(c)
2.0000	3.0000	2.5000	-1.8750
2.5000	3.0000	2.7500	0.6719
2.5000	2.7500	2.6250	-0.6934
2.6250	2.7500	2.6875	-0.0344
2.6875	2.7500	2.7188	0.3127
2.6875	2.7188	2.7031	0.1377
2.6875	2.7031	2.6953	0.0512
2.6875	2.6953	2.6914	0.0083
2.6875	2.6914	2.6895	-0.0131
2.6895	2.6914	2.6904	-0.0024
2.6904	2.6914	2.6909	0.0030
2.6904	2.6909	2.6907	0.0003

Root for Equation 1 found at: 2.6907
octave:2> lab_1_eq_1

Figure 1: Octave Terminal Output and Iteration Table for Equation 1

a	b	c	F(c)
1.0000	2.0000	1.5000	-0.7247
1.5000	2.0000	1.7500	0.1398
1.5000	1.7500	1.6250	-0.3722
1.6250	1.7500	1.6875	-0.1366
1.6875	1.7500	1.7188	-0.0035
1.7188	1.7500	1.7344	0.0668
1.7188	1.7344	1.7266	0.0313
1.7188	1.7266	1.7227	0.0138
1.7188	1.7227	1.7207	0.0051
1.7188	1.7207	1.7197	0.0008

Root for Equation 2 found at: 1.7197
octave:5> |

Figure 2: Octave Terminal Output and Iteration Table for Equation 2

6.2 Graphical Representation

The following graphs illustrate the functions crossing the x-axis, confirming our initial guesses and the roots obtained via the Bisection method.

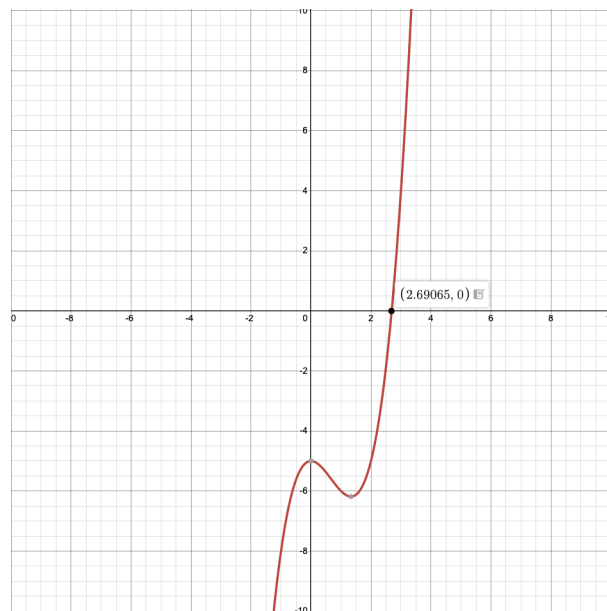


Figure 3: Plot of Equation 1: $f(x) = x^3 - 2x^2 - 5$

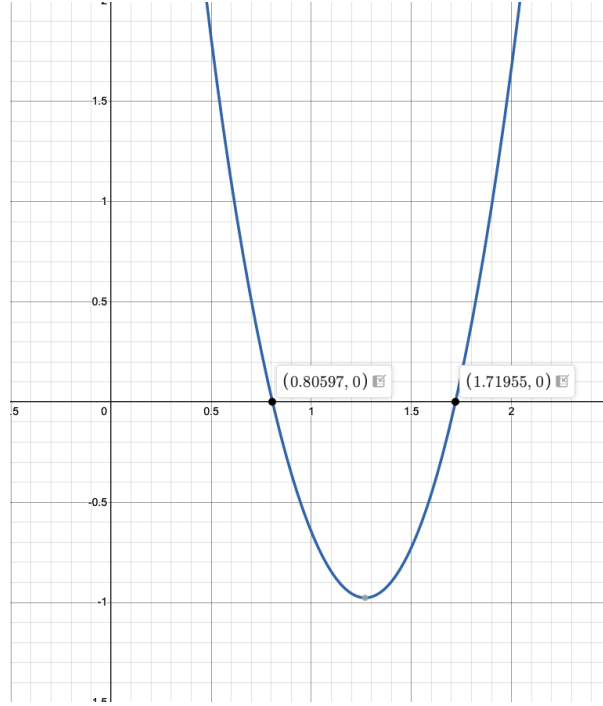


Figure 4: Plot of Equation 2: $f(x) = e^x - 4 \sin(x) - 3 \ln(x)$

7 Discussion

In this experiment, the Bisection method was successfully implemented using independent Octave scripts. By relying on the Intermediate Value Theorem, the algorithm brackets the root and systematically halves the interval.

For the polynomial equation ($x^3 - 2x^2 - 5 = 0$), the initial guesses $a = 2$ and $b = 3$ accurately bounded the true root. Similarly, the transcendental equation involving logarithmic, trigonometric, and exponential functions ($e^x - 4 \sin(x) - 3 \ln(x) = 0$) was successfully solved using initial guesses $a = 1$ and $b = 2$. The method correctly converged within the defined tolerance limit of $lim = 0.001$ in both cases. Although the method guarantees convergence, it requires more iterations compared to advanced methods like Newton-Raphson.

8 Conclusion

The experiment demonstrated the practical implementation of the Bisection method for finding the roots of continuous non-linear equations. By translating mathematical theory into functional Octave scripts, we visualized and quantified the root-finding process, confirming that computational methods offer reliable numerical solutions within a defined margin of error.

References

- [1] Department of Electrical & Computer Engineering, RUET. *Lab Manual: Discrete Mathematics & Numerical Methods (ECE 2214)*. RUET.